

Java – Beautifully Broken

Java finally learns to read JSON, and Oracle's patch calendar goes monthly

Issue #3

Wed, 07 Oct 2026

~32 min read

Level 3/5

Today's two stories

1. **JEP 540, a "Simple JSON API", is now targeted to JDK 28.** Twelve years after the first attempt was shelved, the JDK will ship its own small, strict JSON parser and generator, as an incubator module you switch on with one flag. It won't replace Jackson, and it isn't trying to.
2. **Oracle now ships Java security fixes monthly, not just quarterly.** The first "Critical Security Patch Update" landed on 18 August, the next is due on 17 November, and Azul, Amazon Corretto and Eclipse Temurin have followed. Your patch calendar, and possibly your version-parsing code, needs to keep up.
3. **On the radar:** 6 more things that happened, listed at the end.

STORY 1 JEP 540: the JDK gets a JSON API, twelve years late

JDK 28 · OPENJDK · CORE-LIBS · TARGETED TO JDK 28 (2 OCT 2026) · ~13 MIN READ

Try this the next time you start a small Java project. You want to call a REST endpoint and read one number from the response. Python does it with a module that ships with the language. Go does it with a package in its standard library. In Java, your first step is to open the build file and add a dependency, because the platform that runs a huge share of the world's back ends has never been able to read JSON on its own. JEP 540 changes that. It's a deliberately small API, and the story of why it's small is as interesting as the API itself.

WHAT HAPPENED

On 2 October, Oracle's Inside.java site announced that **JEP 540, "Simple JSON API (Incubator)", is targeted to JDK 28.**¹ The JEP had been Proposed to Target since mid-August, when InfoQ described it as an API that "will ship in the incubating `jdk.incubator.json` module, where it can change incompatibly or be removed based on developer feedback."² InfoQ's JDK 28 overview sums up its purpose as "a simple, standard API for parsing and generating JSON documents without the need for an external library."³

JDK 28 is the next feature release, scheduled for general availability in March 2027.³ So this isn't something you can put in production next week. But it's now on the plan, and the shape of the API is settled enough to understand.

THE BACKGROUND YOU NEED

JSON, and how Java has always handled it

JSON *≈ JavaScript Object Notation: a text format for structured data, made of objects, arrays, strings, numbers, booleans and null* is the default format for web APIs, configuration files and messages between services. Its rules are written down in **RFC 8259** *≈ the internet standard that defines exactly what valid JSON text is*.

The JDK can read XML, ZIP files and HTTP responses, but not JSON. For more than two decades, Java developers have instead relied on third-party libraries such as Jackson, Gson, JSON-P and JSON-B.⁴ Those libraries mostly offer two styles of work. The first is **data binding** *≈ automatically converting JSON into instances of your own Java classes and back, for example a JSON user record into a User object*, which is what Spring does with your `@RequestBody` parameters. The second is a **tree model** *≈ an in-memory representation of a JSON document as a tree of nodes (objects, arrays and plain values) that you navigate by key or index*, where you parse the text and walk through it. Big libraries also offer **streaming parsing** *≈ reading JSON token by token without building the whole document in memory, which is needed for very large inputs*.

The JVM Weekly newsletter put the situation bluntly: "For twelve years the platform couldn't decide whether parsing JSON was any of its business, and in that time Jackson and Gson became part of Java in every sense except the formal one."⁵

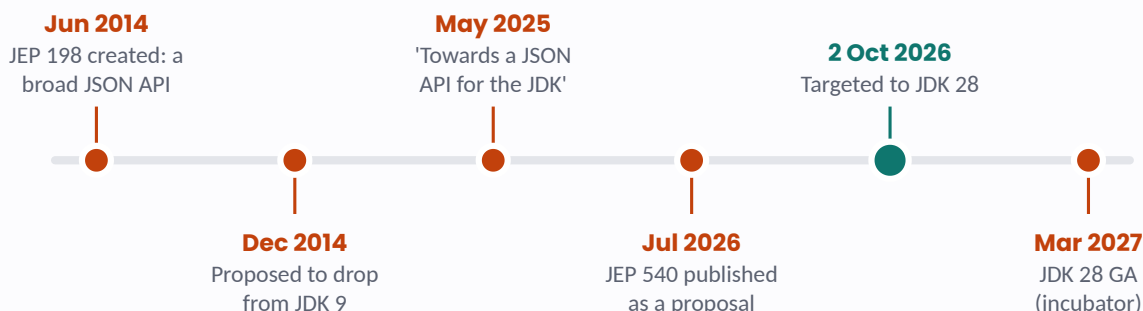
The first attempt, and why it was shelved

The twelve years start in June 2014, when Mike Duigou created **JEP 198, "Light-Weight JSON API"**, reviewed by Brian Goetz. Its goals were broad: parsing and generation per the JSON standard of the time, several parsing styles (token streams, event streams and immutable trees), a builder API, a generator API and even a "transformer" for making modified copies of trees.⁶

In December 2014, Mark Reinhold, the chief architect of the Java platform, proposed dropping it from JDK 9. A JSON API would be useful, he wrote, but "it's not as important as the other features that Oracle is funding, or considering funding, for JDK 9." He added that it could be reconsidered later, especially if future language features such as value types and generics over primitive types made a simpler and more capable design possible.⁷

That last remark turned out to be the key. The Java language of 2014 had no records, no sealed types and no pattern matching. The Java of 2026 has all three, and they change what a JSON API needs to look like.

Java's long road to built-in JSON



Sources: JEP 198; Reinhold, jdk9-dev, 4 Dec 2014; Sandoz, core-libs-dev, 15 May 2025; ADTmag, 28 Jul 2026; Inside.java; InfoQ.

The restart: "batteries included"

On 15 May 2025, Paul Sandoz of Oracle posted "Towards a JSON API for the JDK" to the core-libs-dev mailing list. His argument was that Java follows a "batteries included" philosophy, so developers should be able to handle common data formats without third-party dependencies, while existing libraries remain the right tool for heavier work.⁸

The design he described is the ancestor of JEP 540. JSON is modelled as an **algebraic data type** *≈ a type defined as a fixed set of alternatives, e.g. "a JSON value is exactly one of: object, array, string, number, boolean or null"*: a family of interfaces called `JsonValue`, `JsonArray`, `JsonObject`, `JsonNumber`, `JsonString`, `JsonBoolean` and `JsonNull`. Anything fancier would be built on top: "Advanced features, such as data binding or path-based traversal should be possible to implement as layered features, but for simplicity are not included in the core API."⁸ The prototype lived in a `json` branch of the JDK sandbox repository and, Sandoz reported, passed all but two cases of the `JSONTestSuite` conformance tests. The two failures were deliberate: the API rejects objects with duplicate member names.⁸

A year later, in July 2026, JEP 540 appeared as a formal proposal.⁴ InfoQ describes it as narrower than the JEP 198 plan, concentrating on an immutable, in-memory tree of values and leaving object mapping, streaming, schema validation and advanced customisation to existing libraries.²

HOW IT WORKS

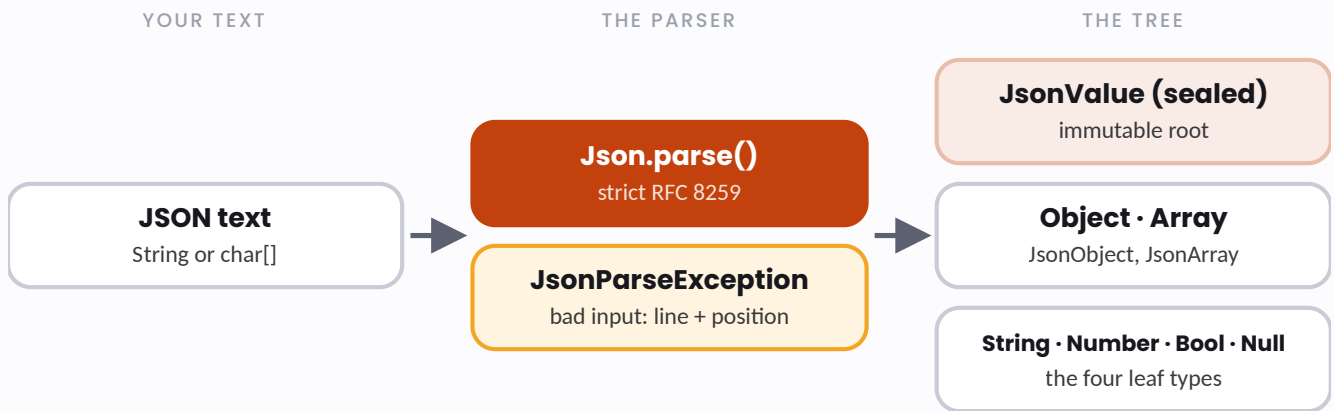
One entry point, one tree of values

Everything starts with the `Json` class. `Json.parse(...)` takes a `String` (or a `char[]`) and gives you back a `JsonValue`: the root of an **immutable** *≈ cannot be changed after it's created, so it's safe to share and cache* tree.

Instances are also **thread-safe** *≈ safe to use from several threads at once without extra locking*.²

`JsonValue` is a **sealed interface** *≈ an interface that lists exactly which types may implement it, so the compiler knows every possible case (Java 17+)*. It permits six subinterfaces, one for each kind of JSON value. Those six are themselves non-sealed, which leaves room for alternative implementations, an idea Sandoz mentioned in 2025 for efficient non-text encodings.^{2,8}

What `Json.parse()` gives you



Type names from InfoQ's summary of the JEP and the Inside Java Newscast #114.

Navigating: `get`, `tryGet` and the `as...` methods

You walk the tree with `get(String)` for an object member and `get(int)` for an array element. When you reach the value you want, you convert it with `asInt()`, `asLong()`, `asDouble()`, `asBoolean()` or `asString()`. Whole containers convert with `asMap()` and `asList()`. Here is the example from the JEP, as shown by InfoQ, reading a temperature from a weather-forecast response (right), next to the tree-model route most teams use today (left). The left pane uses Jackson 2's `ObjectMapper`, which needs the `jackson-databind` dependency; the right pane needs nothing but the JDK.²

Read one number from a JSON response

Today: Jackson's tree model

```
// + jackson-databind dependency
ObjectMapper mapper =
    new ObjectMapper();
JsonNode root = mapper.readTree(body);
int temperature = root
    .get("properties")
    .get("periods").get(0)
    .get("temperature")
    .asInt();
```

JDK 28: JEP 540 (incubator)

```
// --add-modules jdk.incubator.json
int temperature = Json.parse(body)
    .get("properties")
    .get("periods")
    .get(0)
    .get("temperature")
    .asInt();
```

Right-hand code from the JEP example as reported by InfoQ. It needs a JDK 28 build that contains the module; we could not compile it in our sandbox.

The interesting design decisions are in how the API behaves when the data isn't what you expect.

Missing vs null. JSON makes a difference between a member that isn't there and a member that is there with the value `null`. The API keeps that difference. `tryGet(String)` returns an **Optional** `≈ java.util.Optional`: a box that either holds a value or is empty, used instead of returning `null` that's empty when the member is absent, and holds a

`JsonNull` when the member is explicitly null. A second method, `tryValue()`, turns `JsonNull` into an empty `Optional` too, for when you don't care about the distinction.²

Numbers are strict. JSON has a single "number" type with no size limit and no difference between integers and decimals. Java has `int`, `long`, `double` and `BigDecimal`. The API refuses to guess: "`asInt()` and `asLong()` require an exact integral value within the destination type's range." So `3.5`, or a value bigger than `Integer.MAX_VALUE`, won't quietly become something else. `asDouble()` is the exception: it converts any number to a finite `double`, "but may round or lose precision."² Sandoz flagged numbers as a known source of interoperability problems in 2025, and his prototype also exposed the raw text of a number and a `BigDecimal` conversion.⁸ Numeric conversions are one of the areas where the JEP explicitly asks for feedback.²

Two kinds of failure. Broken JSON text throws `JsonParseException`. It's an **unchecked exception** *≈ an exception that extends `RuntimeException`, so the compiler doesn't force you to catch or declare it*, and it records the zero-based line and position of the error. Asking for the wrong type, say `asInt()` on a string, throws `JsonValueException` instead.²

Strict by design

The parser accepts exactly what RFC 8259 allows, and nothing else. There are no comments, no trailing commas and no other syntax extensions. Duplicate member names, such as `{"id": 1, "id": 2}`, are always an error.^{2,9} A duplicate key is ambiguous (which value wins?), and two systems reading the same document might answer differently. Rejecting duplicates outright is, as InfoQ puts it, meant to mitigate those interoperability risks.²

Building and writing JSON

To produce JSON, you build values with factory methods and turn them into text:²

```
JsonObject document = JsonObject.of(Map.of(
    "service", JsonString.of("web_server"),
    "id", JsonNumber.of(3),
    "active", JsonBoolean.of(true)
));
String compact = document.toString();
String pretty  = Json.toDisplayString(document);
```

`toString()` produces compact JSON; `Json.toDisplayString(...)` produces a formatted version meant for humans.² The JEP itself calls construction ergonomics "one likely area for feedback", which is a polite way of admitting that wrapping every value in `JsonString.of(...)` is wordy.²

Pattern matching does the rest

Because `JsonValue` is sealed, it works naturally with **pattern matching for switch** *≈ a switch that branches on the type of a value and binds it to a variable in one step (Java 21+)*. This is how you handle a field that might arrive as a

number or a string:²

```
long id = switch (json.get("id")) {  
    case JsonNumber number -> number.asLong();  
    case JsonString string -> Long.parseLong(string.asString());  
    default -> throw new JsonValueException("Unexpected id type");  
};
```

Sandoz's 2025 post explains why the API looks so minimal: it's betting on the language. As [deconstruction patterns](#) *≈ a planned pattern-matching feature that pulls the components out of an object in a single pattern, the way record patterns do for records* mature, extracting data from JSON should get more elegant without needing extra convenience methods in the API.⁸

Switching it on

Incubator modules aren't visible by default. On the [class path](#) *≈ the traditional list of JARs and directories the JVM loads classes from, as opposed to the module path*, you add the module explicitly, when compiling and when running:^{2,10}

```
javac --add-modules jdk.incubator.json App.java  
java --add-modules jdk.incubator.json App
```

The JVM prints a warning whenever an incubator module is resolved, at compile time, link time and run time, and the run-time warning can't be suppressed.¹⁰

JEP 540 (JDK 28)

- Parse to an immutable tree, navigate, convert
- Build values and write compact or pretty text
- Strict RFC 8259: no comments or trailing commas
- Duplicate member names are an error
- No dependency, but an incubator flag

VS

Jackson, Gson and friends

- Data binding to and from your own classes
- Streaming for very large documents
- Lenient modes, custom formats, modules
- Mature, battle-tested, everywhere already
- A dependency you must keep patched

WHAT IT MEANS FOR YOU

It won't replace Jackson in your Spring Boot service, and it isn't meant to. The JEP leaves out data binding, streaming, lenient parsing and syntax extensions on purpose. Its target is "common tasks such as reading a configuration file, inspecting a REST response, or generating a small JSON payload."² Spring will keep turning request bodies into your DTOs with its own machinery. Nothing in your existing code changes.

Where it will matter is in the places where a dependency feels excessive. Think of a build script, a small CLI tool, a test helper that checks one field in a response, a library that wants to read one config file without pulling Jackson into every user's dependency tree, or the JDK's own tools. Each of those today either adds a library or hand-rolls string parsing. A JDK-provided parser removes that choice. It also removes a dependency

you have to keep patching, which matters when a widely used JSON library has a security advisory and every project that uses it has to upgrade.

The reception has been mixed. The JEP justifies itself by saying, in JVM Weekly's quotation, that "the Python or Go code for these tasks is simple and the Java code should be equally simple." One Hacker News commenter answered that "the examples actually show a lot of ceremony", and JVM Weekly notes that this critique stings precisely because of that goal.⁵ Look at the construction example above and you can see the point. That's exactly why it's an incubator: the shape can still change.

Try it when it lands

Once a JDK 28 early-access build includes the module, a single-file program is the quickest way to get a feel for the API. Save a JSON file, parse it with `Json.parse(Files.readString(path))`, walk it with `get(...)`, and then deliberately break it: add a trailing comma, a comment, a duplicate key, or a `3.5` where you call `asInt()`. Seeing which exception you get, and where it points, tells you more about the API's philosophy than any summary. Feedback goes to the `core-libs-dev` mailing list.

Watch out for

- **Incubator means unstable.** The API can change incompatibly or be removed in a later release.² Incubator APIs live under `jdk.incubator.*`, never in `java.*`, so they're not part of the Java SE standard yet.¹⁰ Don't build a public library API on top of it.
- **Startup warnings.** Every run with the module resolved prints an incubator warning that you can't switch off at run time.¹⁰ Fine for tools, awkward for anything that parses its own console output.
- **Strictness can bite.** If your config files contain comments or trailing commas, or if an upstream API sends duplicate keys, this parser will throw where a more lenient one wouldn't have.²
- **Big numbers.** `asDouble()` may round; `asInt()` / `asLong()` throw unless the value fits exactly.² For money or very large IDs, check the final API for an exact (text or `BigDecimal`) route before you rely on it.
- **Not for huge files.** There's no streaming. The whole document is parsed into memory.²

WHAT HAPPENS NEXT

The JEP asks for feedback in four areas: the navigation model, exception semantics, numeric conversions and construction ergonomics.² Incubator APIs normally stay in incubation for "a small number of feature releases" before they're either made permanent or removed.¹⁰ JDK 28 itself is due in March 2027.³ So the realistic timeline is that you'll be able to experiment in 2027, and a final, standard `java.*` JSON API is a few releases beyond that, if the feedback is good. After twelve years, a few more releases to get it right doesn't seem like much to ask.

Words from this story

JSON: a text format for structured data, made of objects, arrays, strings, numbers, booleans and null.

RFC 8259: the internet standard that defines exactly what valid JSON text is.

Data binding: automatically converting JSON into your own Java classes and back.

Tree model: an in-memory tree of JSON nodes that you navigate by key or index.

Streaming parsing: reading JSON token by token without loading the whole document into memory.

Algebraic data type: a type defined as a fixed set of alternatives.

Immutable: cannot be changed after it's created.

Thread-safe: safe to use from several threads at once without extra locking.

Sealed interface: an interface that lists exactly which types may implement it (Java 17+).

Optional: `java.util.Optional`, a box that either holds a value or is empty.

Unchecked exception: an exception extending `RuntimeException`, which you don't have to catch or declare.

Pattern matching for switch: a switch that branches on a value's type and binds it to a variable (Java 21+).

Deconstruction patterns: a planned feature for pulling an object's components out in a single pattern.

Class path: the traditional list of JARs and directories the JVM loads classes from.

Incubator module: a `jdk.incubator.*` module holding an experimental API that may change or be removed (JEP 11).

For years, Java security had a rhythm you could put in a calendar: four times a year, on a Tuesday in January, April, July and October, Oracle published its Critical Patch Update and every JDK vendor followed. Ops teams planned their quarters around it. That rhythm just broke. In July, Oracle's quarterly update across all its products contained more patches than ever before. In August, Java got its first monthly security release. And the reason, according to Oracle, is the same one that pushed Spring onto a monthly train: AI is finding vulnerabilities faster than a quarterly calendar can absorb.

WHAT HAPPENED

On 20 July, Donald Smith, Oracle's vice president of product management, published on the Java blog "Transitioning Java to more frequent security updates". Its summary is short: "Oracle intends to provide Java security updates more frequently than the current quarterly CPU cadence of January, April, July, and October."¹¹

The new releases are called **Critical Security Patch Updates (CSPUs)**. They're "intended to provide an opportunity every month for targeted fixes that address high-priority vulnerabilities, instead of having to wait until the next quarterly CPU."¹¹ The first one shipped on **18 August 2026**. Oracle updated the post on 10 September to say the next planned Java CSPU is on **17 November 2026**, alongside the regular October and January quarterly updates, with "multiple" monthly updates planned for 2027.¹¹

The August Java CSPU covered six release lines at once: 26.0.2.1, 25.0.4.1, 21.0.12.1, 17.0.20.1, 11.0.32.1 and 8u503.¹² Oracle's advisory listed six new security patches for Java SE, four of them exploitable remotely without authentication, with a highest CVSS base score of 8.2.¹³ Tenable's scanner entry for it names the affected components as the installer, **JSSE** ≈ *the Java Secure Socket Extension: the JDK's implementation of TLS, the protocol behind HTTPS* and networking.¹⁵

The rest of the ecosystem moved with it. Azul had already committed on 23 July to monthly CSPUs for its builds, starting in August, on the third Tuesday of each month.¹⁷ Amazon released Corretto CSPU builds the same day as Oracle, 18 August.¹⁹ Eclipse Temurin, the Adoptium project's free OpenJDK build, followed on 2 September with what it called "a security-only, out-of-band release addressing vulnerabilities disclosed by Oracle."²⁰

THE BACKGROUND YOU NEED

CPUs: Java's quarterly heartbeat

A **Critical Patch Update (CPU)** ≈ *Oracle's quarterly, cumulative security release for all its products, on the third Tuesday of January, April, July and October* bundles every security fix Oracle has ready for its products into one release.¹⁸ For Java, a CPU comes out as a new update version of each supported JDK line. Each fix is tracked as a **CVE** ≈ *Common Vulnerabilities and Exposures: a public ID such as CVE-2026-61308 that names one specific security flaw* and scored with **CVSS** ≈ *Common Vulnerability Scoring System: a 0–10 severity score, where 7.0+ is "high" and 9.0+ is "critical"*.

Why does this matter to OpenJDK builds that aren't Oracle's? Because security fixes for OpenJDK are coordinated privately among vendors and released on the same day. That's why Microsoft's July OpenJDK update, for example, points to the "OpenJDK Vulnerability Advisory July 2026" and shipped as 25.0.4, 21.0.12, 17.0.20 and 11.0.32: the same update numbers as Oracle's.²² When Oracle's calendar changes, the whole ecosystem's calendar changes.

The fourth digit

Java version numbers have four parts, defined by JEP 322 back in JDK 10: `$FEATURE.$INTERIM.$UPDATE.$PATCH`. The `$UPDATE` counter is "incremented for compatible update releases that fix security issues, regressions, and bugs". The fourth, `$PATCH`, is "the emergency patch-release counter, incremented only when it's necessary to produce an emergency release to fix a critical issue."²³ So `21.0.12` is the twelfth update of JDK 21. Most developers have rarely seen a fourth digit at all.

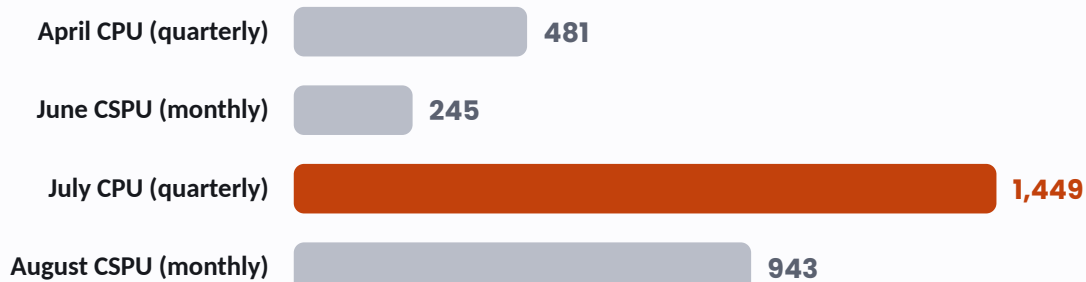
CSPUs turn that rarity into routine. As JVM Weekly put it, the fourth digit was "previously reserved for emergency releases but is now used for CSPUs."⁵ `21.0.12.1` means "21.0.12 plus one round of urgent security fixes".

Why now: the AI patch flood

Oracle's explanation is that "the latest generation of AI is transforming how software vulnerabilities are identified and fixed, increasing the speed and scale of discovery and remediation."¹¹ Azul's CEO Scott Sellers made the same point when announcing its monthly programme: "As AI sharply increases the volume of threats enterprises face, enterprises shouldn't have to choose between the two", meaning speed and stability.¹⁷

The numbers across Oracle's whole product portfolio show the pressure. According to Tenable, Oracle introduced monthly CSPUs for its other products in May 2026. The April CPU contained 481 patches; the July CPU, 1,449. Even the "small" monthly releases are big: 245 patches in June and 943 in August.¹⁶ JVM Weekly compared July's total with 309 patches a year earlier.⁵

Security patches per Oracle release, all products (2026)



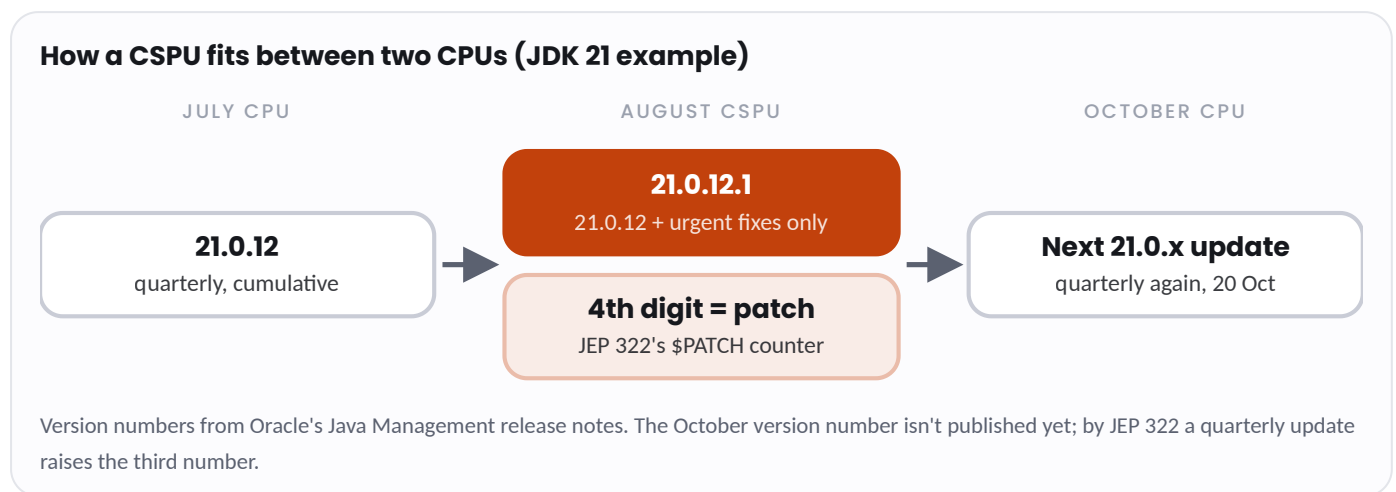
Source: Tenable, 18 Aug 2026. Counts are for all Oracle products (databases, middleware, apps and more), not Java alone. Java SE got six patches in the August CSPU per Oracle's advisory.

Java is a small slice of those totals. But Java security fixes are released as part of Oracle's programme, so Java inherits its calendar.

HOW IT WORKS

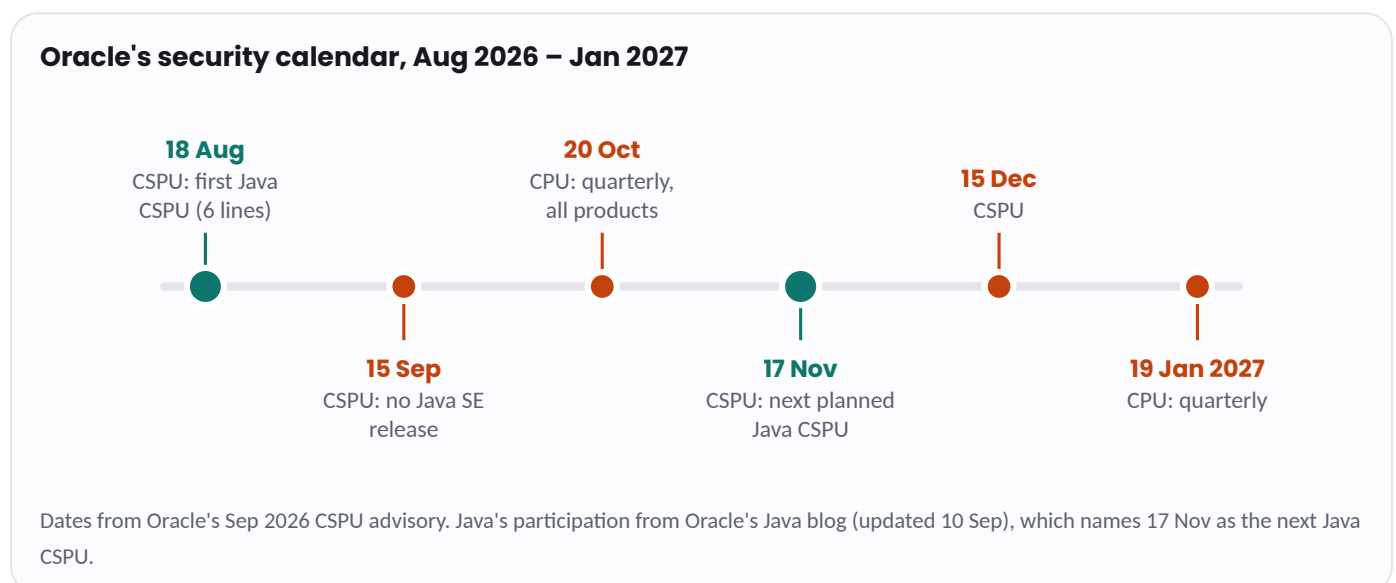
A CSPU is a small branch off the last CPU

A CSPU isn't a mini-CPU with everything in it. Azul's Frank Delporte describes CSPUs as containing security fixes and stability patches only: no new features, no API changes and no behaviour changes beyond fixing the vulnerability. They branch from the last quarterly release tag, apply the fixes with minimal scope, then merge back, "leaving the regular quarterly cycle unaffected."¹⁸ Oracle's own advisory calls a CSPU a release that "provides targeted, high-priority security fixes in a smaller, more focused format, making them easier to apply with minimal disruption", which complements the quarterly cumulative CPUs.¹³ Temurin describes it the same way: "a security update based off of the quarterly 'CPU' releases."²⁰



The calendar: third Tuesday, every month

Oracle's September advisory spells out the new rule: "Security patches are released on the third Tuesday of each month."¹⁴ Its published schedule:



Note the important detail: a monthly *slot* doesn't guarantee a monthly *Java release*. September's CSPU contained fixes for Oracle GraalVM builds but no Java SE release,¹⁴ and Oracle's Java post lists November as the next Java CSPU after August.¹¹ Azul describes CSPUs as inserted between quarters "when urgency warrants", not guaranteed every month.¹⁸ Think of it as a monthly window that Java uses when it needs to.

Every vendor, slightly different

Because the fixes are coordinated, every major OpenJDK vendor shipped an August CSPU, but each with its own version string and timing:

Vendor	JDK 21 build	Released
Oracle JDK	21.0.12.1	18 Aug ¹²
Amazon Corretto	21.0.12.9.1	18 Aug ¹⁹
Eclipse Temurin	21.0.12.1+1	2 Sep ²⁰
Azul Zulu	CSPUs on the third Tuesday from Aug	23 Jul announcement ¹⁷

Corretto adds its own build number before the patch digit, and Temurin's Java 8 CSPU is 8u504 while Oracle's is 8u503.^{19,20,12} Temurin's announcement says it is "easing into a more regular 'CSPU' release cadence", which explains its two-week lag the first time round.²⁰

WHAT IT MEANS FOR YOU

Your patch cadence probably needs to become monthly. Oracle says it plainly: "Users need to patch their Java installations more often to remain on the Java security baseline. Organizations that deliver applications bundled with a JDK or JRE may also need to assess how a faster update cadence affects their build, test, release, and deployment processes."¹¹ If you build container images on a base JDK image, that means rebuilding and redeploying on the CSPU dates too, not just four times a year.

Treat a CSPU like any other patch. Azul's advice is to handle CSPUs exactly as you handle quarterly CPUs: let your dependency tooling pick them up, run the existing test suite, and push them through your normal pipeline.¹⁸ Because they are security-only and narrow, they're low-risk to apply. The real work is making your pipeline cheap enough that monthly runs don't hurt.

One shared patch week. This lines up neatly with the story from our previous issue. Spring's monthly "Patch Thursday" falls on "the Thursday after the third Monday of the month";³¹ Oracle's CSPU falls on the third Tuesday. In October that's 20 October for Oracle's CPU and 22 October for Spring's first train; in November, 17 November for the next Java CSPU and 19 November for Spring.^{14,31} For a typical Spring Boot service, the third week of the month becomes "patch week" for both the JDK and the framework.

Check anything that parses Java versions. Code and scripts that assume a version has three numbers will trip on `21.0.12.1`, and Corretto's longer strings stretch that further.¹⁹ In Java code, use `Runtime.version()`, which already exposes all four parts, including `patch()`, as defined by JEP 322.²³ In shell scripts, Dockerfiles and monitoring dashboards, look for regular expressions such as `\d+\.\d+\.\d+\$`.

In the real world: the installer that didn't upgrade

The first Temurin CSPU produced a small, instructive bug report. A user on an Apple Silicon Mac installed the `jdk-25.0.4.1+1` package and found the JDK still reported `25.0.4+7`. The cause: the existing 25.0.4 installation was still present and the package didn't overwrite it cleanly; removing the old directory and reinstalling fixed it. Maintainers also noted they hadn't published an official GA build for that configuration because of test failures.²¹ The lesson for monthly patching: after every rollout, check what `java -version` actually reports, rather than trusting that the package manager did its job.

JDK 21 users on Oracle's free licence: this is the moment. Oracle's August blog says JDK 21 updates "through and including September 2026" are available under the free **NFTC** *≈ Oracle's No-Fee Terms and Conditions licence, which allows free production use of Oracle JDK builds for a limited period*, and that updates beginning with the October 2026 CPU are planned under the **OTN** *≈ the Oracle Technology Network licence, which allows free development use but generally requires a paid subscription for production* licence.²⁴ Since September brought no Java CSPU, the August CSPU build 21.0.12.1 looks to be the last Oracle JDK 21 update under NFTC. From 20 October, staying patched on Oracle JDK 21 means a subscription, moving to JDK 25 (whose NFTC updates are planned until October 2028²⁴), or switching to another vendor's build. Issue #1 covered those options in detail.

Watch out for

- **"Monthly" isn't guaranteed.** September had a CSPU date but no Java SE release.¹⁴ Subscribe to your vendor's advisories instead of assuming a fixed rhythm.
- **Vendors don't all ship on the same day.** Temurin's first CSPU came two weeks after Oracle's.²⁰ If you rely on a specific distribution, watch its release page, not Oracle's.
- **Version strings differ by vendor.** `21.0.12.1` (Oracle), `21.0.12.9.1` (Corretto), `21.0.12.1+1` (Temurin) are all the same security level.^{12,19,20}
- **Non-LTS lines get CSPUs too, briefly.** August's release included JDK 26 (26.0.2.1),¹² but Oracle's roadmap ends JDK 26 support in September 2026.²⁵ If you're on a non-LTS release, you still need to move to the next one.
- **The quarterly CPU still matters most.** CSPUs are narrow. Bug fixes and broader changes still arrive in the quarterly CPUs, the next on 20 October.^{18,14}

WHAT HAPPENS NEXT

The next dates to circle are **20 October** (the quarterly CPU, and the switch of Oracle JDK 21 updates to the OTN licence) and **17 November** (the next planned Java CSPU), followed by 15 December and the 19 January 2027 CPU.^{14,11} Oracle expects "multiple" monthly Java updates in 2027.¹¹ The open question is how often Java will actually need its monthly slot. If AI-assisted vulnerability research keeps accelerating, the honest answer may soon be "most months", and the teams that automated their JDK upgrades this autumn will be glad they did.

Words from this story

Critical Patch Update (CPU): Oracle's quarterly, cumulative security release, on the third Tuesday of January, April, July and October.

Critical Security Patch Update (CSPU): Oracle's new monthly, targeted security release between CPUs, versioned with a fourth digit (e.g. 21.0.12.1).

CVE: Common Vulnerabilities and Exposures, a public ID naming one specific security flaw.

CVSS: Common Vulnerability Scoring System, a 0–10 severity score.

JSSE: the Java Secure Socket Extension, the JDK's implementation of TLS.

\$PATCH: the fourth part of a Java version number, defined by JEP 322 as the emergency patch-release counter.

NFTC: Oracle's No-Fee Terms and Conditions licence, allowing free production use of Oracle JDK for a limited period.

OTN: the Oracle Technology Network licence, free for development but generally requiring a subscription for production use.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **JEP 542, PEM encodings go final in JDK 28:** after three previews in JDK 25–27, the API for reading and writing keys and certificates in PEM text is targeted to JDK 28, with `DEREncodable` renamed to `BinaryEncodable`.^{26,1}
- **JEP 541, macOS/x64 port deprecated for removal:** targeted to JDK 28, it starts the end of Oracle's Intel-Mac JDK port, since Apple has moved its hardware to AArch64.^{1,27}
- **Cheaper `Arena.ofConfined()` in JDK 28:** the Foreign Function & Memory API now serves small native allocations from per-thread pools, making 5-byte allocations 6.8–18.6x faster with no code changes.²⁸
- **Faster post-quantum crypto:** CPU intrinsics in JDK 27 and 28 speed up ML-KEM by up to 218%, with similar gains for ML-DSA, on Linux x64 and AArch64.²⁹
- **JDK 27 performance round-up:** Oracle's summary highlights compact object headers and G1 by default, plus `HashMap` bulk operations 61–86% faster, from over 2,300 commits.³⁰
- **Spring's September milestone wave:** Spring AI 2.1.0-M1, Spring Data 2026.1.0-M2, Spring Batch 6.1.0-M2 and Spring Integration 7.2.0-M2 joined Boot 4.2.0-M2 ahead of November's releases.³²

1. **Inside.java** home page listing: "JEP targeted to JDK 28: 540" (2 Oct), "541" (29 Sep) and "542" (6 Oct), checked 7 Oct 2026.
2. **JEP 540 Proposed to Target JDK 28 with a Simple JSON API**, A N M Bazlur Rahman, InfoQ, 17 Aug 2026.
3. **JDK 27 and JDK 28: What We Know So Far**, InfoQ, 24 Aug 2026.
4. **Why Java May Finally Get a Standard JSON API**, John K. Waters, ADTmag, 28 Jul 2026.
5. **The Fourth Digit, a Twelve-Year Silence, and a Plan for When the Dinosaurs Get Out**, JVM Weekly vol. 188.
6. **JEP 198: Light-Weight JSON API**, OpenJDK.
7. **Proposed to Drop: JEP 198: Light-Weight JSON API**, Mark Reinhold, jdk9-dev, 4 Dec 2014.
8. **Towards a JSON API for the JDK**, Paul Sandoz, core-libs-dev, 15 May 2025.
9. **JSON API, Valhalla Progress, LTS ❤️ PQC - Inside Java Newscast #114**, Nicolai Parlog, 15 Aug 2026.
10. **JEP 11: Incubator Modules**, OpenJDK.
11. **Transitioning Java to more frequent security updates**, Donald Smith, Oracle Java blog, 20 Jul 2026, updated 10 Sep 2026.
12. **Oracle Security Critical Patch Update (CSPU) August 2026 for Oracle Java SE**, Oracle Java Management release notes, 18 Aug 2026.
13. **Oracle Critical Security Patch Update Advisory - August 2026**, Oracle, 18 Aug 2026.

14. [Oracle Critical Security Patch Update Advisory - September 2026](#), Oracle, 15 Sep 2026.
15. [Oracle Java SE Multiple Vulnerabilities \(August 2026 CSPU\)](#), Tenable Nessus plugin 338352.
16. [Oracle August 2026 Critical Security Patch Update addresses 925 CVEs](#), Tenable, 18 Aug 2026.
17. [Azul to Deliver Monthly Critical Security Patch Updates for Java Across All Supported LTS Versions](#), Azul, 23 Jul 2026.
18. [Azul Will Deliver Monthly Java Critical Security Patch Updates](#), Frank Delporte, Azul, 23 Jul 2026.
19. [Amazon Corretto August 2026 Critical Security Patch Updates](#), AWS, 18 Aug 2026.
20. [Eclipse Temurin 8u504, 11.0.32.1, 17.0.20.1, 21.0.12.1, 25.0.4.1 and 26.0.2.1 Available](#), Adoptium PMC, 2 Sep 2026.
21. [macOS aarch64 - Temurin jdk-25.0.4.1+1 - really 25.0.4+7 \(issue #4549\)](#), adoptium/temurin-build, Aug 2026.
22. [Java OpenJDK July 2026 Patch & Security Update](#), Microsoft for Java Developers, Jul 2026.
23. [JEP 322: Time-Based Release Versioning](#), OpenJDK (delivered in JDK 10).
24. [JDK 21 approaches end of permissive license](#), Oracle Java blog, 14 Aug 2026.
25. [Oracle Java SE Support Roadmap](#), Oracle.
26. [Java News Roundup \(week of 24 Aug 2026\)](#), InfoQ, 31 Aug 2026.
27. [Java deprecates support for macOS x86](#), OSNews, 27 Jul 2026.
28. [Making Arena.ofConfined\(\) Even Cheaper in JDK 28](#), Per-Ake Minborg, Inside.java, 5 Oct 2026.
29. [Faster Post-Quantum Cryptography with JDK Intrinsic](#), Mark Powers, Inside.java, 30 Sep 2026.
30. [Performance Improvements in JDK 27](#), Per-Ake Minborg and Claes Redestad, Inside.java, 28 Sep 2026.
31. [Releasing Spring for Modern Challenges](#), Michael Minella, spring.io, 21 Sep 2026.
32. [This Week in Spring - September 29th, 2026](#), Josh Long, spring.io.